

UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE PROCESSAMENTO DE ENERGIA ELÉTRICA



Extension Methods e Tratamento de Erros

Curso de Engenharia de Controle e Automação
DPEE1090 - Programação orientada a objetos para automação

Prof. Renan Duarte

1º semestre de 2024

Observação

Extension Methods e tratamento de erros

Os conteúdos desta aula não fazem parte da ementa da disciplina e serão vistos como um “extra” pois são bastante úteis e comuns em aplicações do dia-a-dia.

Na avaliação final, os conteúdos de hoje **não serão cobrados**

Sumário

Extension Methods e Tratamento de erros

Extension Methods

- Definição e utilização

Detecção e tratamento de erros

- Exceções
- Blocos try/catch/finally
- Lançamento de exceções

Extension methods

Definição

São métodos que permitem adicionar novas funcionalidades a tipos existentes (classes, structs, interfaces) sem modificar o código original ou criar novas classes derivadas.

Benefícios:

- **Reutilização de Código:** Permite reutilizar funcionalidades em vários tipos sem duplicação de código.
- **Manutenção:** Facilita a manutenção e a leitura do código ao agrupar funcionalidades relacionadas.
- **Encapsulamento:** Adiciona métodos a classes existentes sem alterar a implementação original, mantendo o encapsulamento.

<https://learn.microsoft.com/pt-br/dotnet/csharp/programming-guide/classes-and-structs/extension-methods>

Extension methods

Suporte

Nem todas as linguagens orientadas a objetos suportam métodos de extensão e algumas utilizam outra nomenclatura para obter a mesma funcionalidade.

- **C#:** Introduziu Extension Methods no C# 3.0, amplamente usado com LINQ.
- **JavaScript:** Usa o prototype para adicionar métodos a tipos existentes.
- **Swift:** Permite extensões em classes, structs, enums e protocolos.
- **Kotlin:** Suporta Extension Functions para adicionar métodos a classes existentes.
- **Ruby:** Permite reabertura de classes para adicionar ou modificar métodos.

Extension methods

Declaração em C#

- Static Class: Extension Methods devem ser definidos dentro de uma classe estática.
- Static Method: O método de extensão também deve ser estático.
- this Keyword: O primeiro parâmetro do método de extensão é precedido pela palavra-chave this e especifica o tipo que está sendo estendido.

`public static class Extensions`

Tipicamente se usa o sufixo "Extensions" após o nome da classe original

`public static void This(int y)`

Tipo do objeto recebido e nome do objeto

Exemplo 1

Determinada aplicação demanda um método que retorna o número de palavras em uma string. Para manter todo código relacionado a strings em um único lugar, pode-se usar extension methods para essa implementação.

```
public static class StringExtensions
```

```
{  
    public static int CountWords(this string s)
```

```
{  
    if (string.IsNullOrEmpty(s))  
        return 0;  
    return s.Split(' ').Length;  
}
```

Exercício 1

Criar um método de extensão da estrutura DateTime para apresentar um objeto DateTime na forma de tempo decorrido desde a data informada, podendo ser em horas (se menor que 24h) ou em dias caso contrário. Por exemplo:

```
Datetime dt = new DateTime(2017, 10, 10, 10, 10, 10);  
Console.WriteLine(dt.Elapsed.ToString());  
Resultado: 00:00:00
```

```
Datetime dt, new DateTime(2017, 10, 10, 10, 10, 10);  
Console.WriteLine(dt.Elapsed.ToString());  
Resultado: 00:00:00
```


Detecção e tratamento de erros

Exceções

Uma exceção é qualquer condição de erro ou comportamento inesperado encontrado por um programa em execução

- No .NET, uma exceção é um objeto herdado da classe `System.Exception`
- Quando lançada, uma exceção é propagada na pilha de chamadas de métodos em execução, até que seja capturada (tratada) ou o programa seja encerrado

Exemplo:

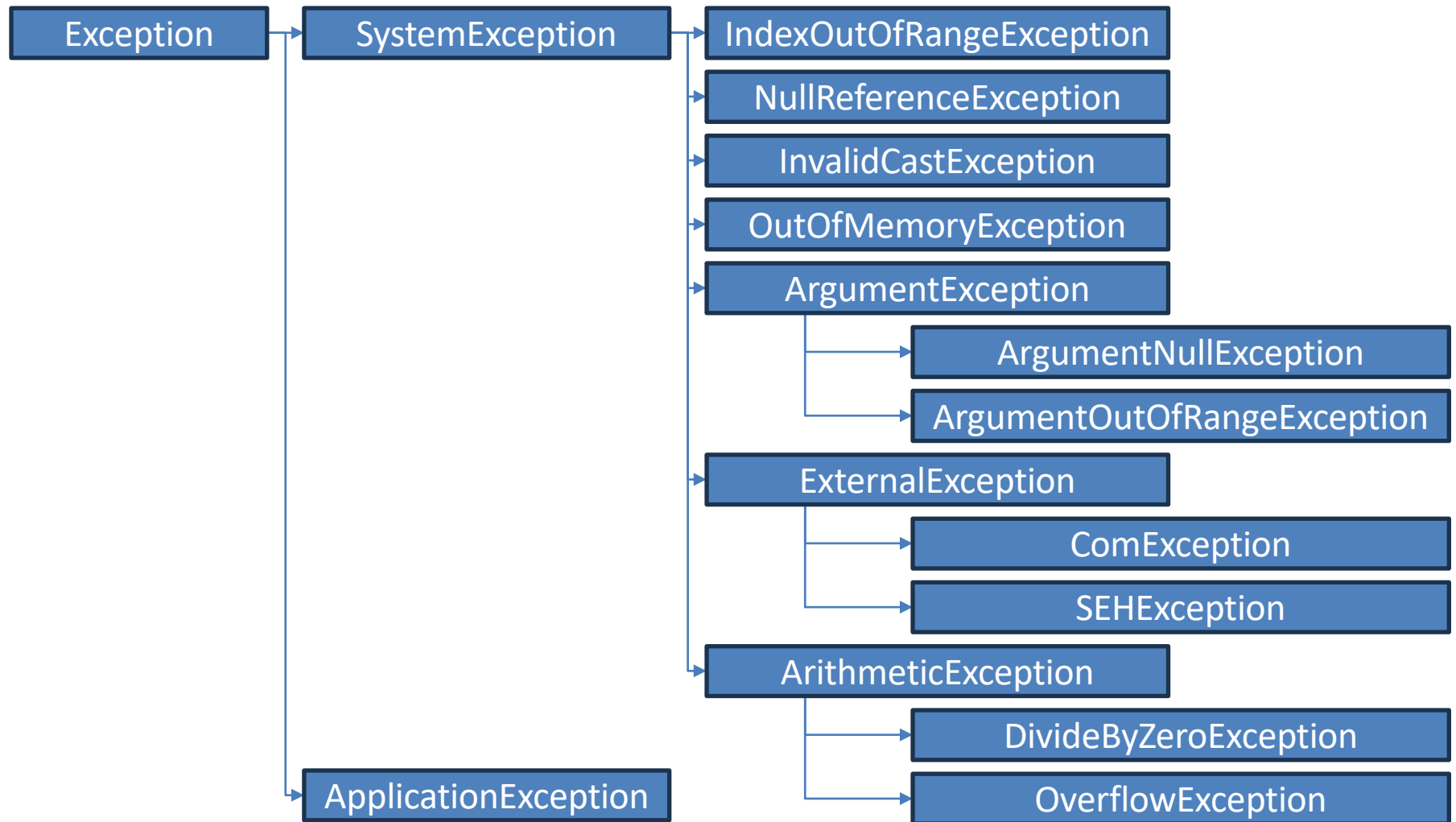
```
int y;  
Console.WriteLine("y = ");
```

Exception Unhandled

System.DivideByZeroException: 'Attempted to divide by zero.'

Detecção e tratamento de erros

Exceções do .NET



Detecção e tratamento de erros

Por que exceções?

O modelo de tratamento de exceções permite que erros sejam tratados de forma consistente e flexível, usando boas práticas

Vantagens

- Delega a lógica do erro para a classe / método responsável por conhecer as regras que podem ocasionar o erro
- Trata de forma organizada (inclusive hierárquica) exceções de tipos diferentes
- A exceção pode carregar dados

<https://learn.microsoft.com/pt-br/dotnet/csharp/language-reference/statements/exception-handling-statements>

Detecção e tratamento de erros

Estrutura *try-catch*

Estrutura de controle usada para tratar exceções do código

Consiste de dois blocos:

- Bloco *try*: Contém o código que representa a execução normal do trecho de código que **pode** acarretar em uma exceção
try
- Bloco *catch*: Contém o código a ser executado caso uma exceção ocorra. Deve ser especificado o tipo da exceção a ser tratada (*upcasting* é permitido)
catch É necessário especificar o tipo da exceção a ser tratada

Exemplo 2

Ler dois números inteiros digitados pelo usuário e calcular o resultado da divisão do primeiro pelo segundo.

`tsy`

```
int n,      int Rasse Cõsõlê RêãđLîê  
int n,      int Rasse Cõsõlê RêãđLîê
```

Cõsõlê WsîťêLîê n, n,

`çắťç` DîwîđêBýŽêsôÉyçêřťîõ êy

Cõsõlê WsîťêLîê Têñťắťîwắ đê dîwîşbộ rộs cêsô

`çắťç` GõsắťÉyçêřťîõ êy

Cõsõlê WsîťêLîê Gõsắťộ ìñwắlîđộ

Detecção e tratamento de erros

Bloco *finally*

É um bloco que contém código a ser executado independentemente de ter ocorrido ou não uma exceção.

Exemplo clássico: fechar um arquivo ou conexão de banco de dados ao final do processamento.

`try`

← Sempre é executado

`except` É executado se ocorrer um erro

← Executado se ocorrer um erro

`finally`

← Sempre é executado

Exemplo 3

Ler o conteúdo de um arquivo .txt e exibir seu conteúdo na tela. Exibir mensagens de erro caso o arquivo não seja encontrado, caso não se tenha permissão para acessar o arquivo ou qualquer outro erro.

Utilize os blocos try-catch-finally para obter o comportamento esperado.

Obs: Esse exemplo demanda conteúdos não aprendidos nesse curso como o *StreamReader* do .NET.

Seu objetivo é apenas demonstrar o uso dos blocos de controle para tratamento de erros

Detecção e tratamento de erros

Lançamento de exceções

Podemos, dentro das nossas classes, lançar exceções para o programa principal quando um erro é detectado. Para isso, usamos o comando **throw**

```
try {  
    // código que pode gerar exceção  
} catch (Exceção e) {  
    // tratamento da exceção  
}
```

Exemplo:

```
try {  
    // código que pode gerar exceção  
} catch (Exceção e) {  
    // tratamento da exceção  
}
```

```
try {  
    // código que pode gerar exceção  
} catch (Exceção e) {  
    // tratamento da exceção  
}
```

```
try {  
    // código que pode gerar exceção  
} catch (Exceção e) {  
    // tratamento da exceção  
}
```


Detecção e tratamento de erros

Exceções personalizadas

São classes de exceção definidas pelo usuário que herdam de uma das classes de exceção base fornecidas pelo .NET (geralmente *ApplicationException*).

Propósito: Usadas para representar erros específicos da aplicação que não são cobertos pelas exceções padrão.

Vantagens:

- Clareza: Fornecem uma descrição mais clara e específica do erro ocorrido.
- Organização: Ajudam a organizar o tratamento de erros em grandes aplicações.
- Manutenção: Facilitam a manutenção e a depuração do código.

Detecção e tratamento de erros

Criação de exceções personalizadas

Passos:

- Herança: Crie uma nova classe que herde de `ApplicationException`.
- Construtores: Implemente construtores para diferentes cenários de inicialização.
- Mensagem: Inclua uma mensagem descritiva e, opcionalmente, outras propriedades.

Usualmente nomeamos essa classe de acordo com o domínio da aplicação, ou seja, ao contexto específico ou ao campo de atuação no qual uma aplicação de software opera, seguido do sufixo “Exception”.



Exemplo 4

Implementar uma funcionalidade que permita realizar saques em uma conta bancária. Se o saldo da conta não for suficiente para o saque solicitado, você deve lançar uma exceção personalizada `SaldoInsuficienteException`.

Para isso, duas classes serão necessárias:

- A classe `ContaBancaria` com atributo `Saldo` e o método `Sacar`
- A classe `ContaBancariaException` que implementa a exceção personalizada

Exercício 2

Desenvolver uma classe de extensão em C# para manipulação de números inteiros. Sua tarefa é implementar um método de extensão (IsPrime) que determine se um número inteiro é primo ou não. Além disso, você deve criar um método principal para testar esta funcionalidade, incluindo o tratamento de exceções.

No método principal, solicite ao usuário que digite um número inteiro para verificar se é primo. Utilize tratamento de exceções para lidar com possíveis erros de entrada, como valores não numéricos ou valores que não são inteiros válidos. Caso ocorra um erro de entrada, exiba uma mensagem de erro amigável e continue solicitando uma entrada válida até que o usuário forneça um número válido.

Revisão

Próxima aula

Exercícios

- Exercícios envolvendo todos os conceitos tratados no semestre

Dúvidas?

renan.duarte@gedre.ufsm.br

GEDRE – Prédio 10 – CTLAB